

School Timetable Distribution Algorithm

الهدف

بناء خوارزمية ذكية لتوزيع الحصص داخل الجدول المدرسي بحيث تقوم بتوليد جدول صالح ومنطقي ومتوازن، اعتمادًا على:

- الفصول
- المواد
- المعلمين
- الفترات الزمنية
- القيود الأكاديمية
- النصاب
- التفضيلات
- جودة التوزيع

الخوارزمية يجب ألا تكون توزيعًا عشوائيًا، بل يجب أن تعمل كالتالي:

Constraint-Based Scheduling + Heuristic Prioritization + Iterative Assignment + Local Conflict Resolution + Optimization Pass

الفكرة الأساسية للخوارزمية (1)

الخوارزمية تحاول حل المعادلة التالية لكل حصة مطلوبة:

فصل + مادة + معلم صالح + وقت صالح + بدون تعارض + ضمن النصاب + ضمن قيود المدرسة

أي أن كل حصة لا تُنشأ إلا إذا تحقق الآتي:

- المادة مطلوبة لهذا الفصل
- يوجد معلم صالح لها
- هذا المعلم متاح
- هذا الفصل متاح
- الفترة الزمنية صالحة
- لا يوجد تعارض
- لا يتم كسر النصاب
- لا يتم كسر القواعد اليومية
- يتم اختيار أفضل توزيع ممكن وليس مجرد أول مكان متاح

نوع الخوارزمية (2)

الخوارزمية المطلوبة هي:

Constraint-Based Heuristic Scheduler

وتعمل عبر 5 طبقات:

1. Preparation Layer
تجهيز البيانات والسياق
2. Demand Modeling Layer
تحويل الاحتياجات إلى وحدات قابلة للجدولة
3. Constraint Filtering Layer
استبعاد كل الاختيارات غير الصالحة
4. Scoring & Assignment Layer
تقييم أفضل الفترات وإسنادها
5. Repair & Optimization Layer
إصلاح التعارضات وتحسين جودة الجدول

المدخلات المطلوبة (3)

الخوارزمية تعمل فقط إذا توفرت البيانات التالية من قاعدة البيانات:

- المدرسة
- العام الدراسي
- الفصل الدراسي
- الأيام الدراسية المفعلة
- الفترات التعليمية
- الفصول
- الصفوف
- المواد
- عدد الحصص الأسبوعية لكل مادة
- ربط المواد بالصفوف
- ربط المعلمين بالمواد
- توفر المعلمين
- توفر الفصول
- قواعد النصاب
- التفضيلات
- قيود المادة
- قيود الغرف إن كانت مفعلة
- إعدادات الجدول الافتراضية

المخرجات المطلوبة (4)

الخوارزمية يجب أن تنتج:

- جدول كامل أو جزئي
- قائمة الحصص الموزعة
- قائمة الحصص غير الموزعة
- عدد التعارضات

- درجة جودة التوزيع
- تحذيرات تشغيلية
- واضحة Diagnostics
- من الجدول Draft نسخة

منطق العمل العام (5)

الخوارزمية تنفذ المراحل التالية بالتسلسل:

1 المرحلة: Load Active Context

تحميل كل السياق النشط من قاعدة البيانات

- المدرسة الحالية
- العام الدراسي الحالي
- الفصل الدراسي الحالي
- الأيام الدراسية النشطة
- الفترات التعليمية فقط
- الفصول النشطة
- المواد المطلوبة لكل صف
- المعلمين المرتبطين بكل مادة
- القيود
- التفضيلات
- النصاب
- النسخة الحالية إذا لزم

الهدف

موحد وواضح Scheduling Context بناء

2 المرحلة: Validate Readiness

قبل البدء، يجب التأكد من أن البيانات كافية لتوليد جدول

يتم التحقق من:

- وجود أيام دراسية
- Teaching Slots وجود
- وجود فصول
- وجود مواد لكل صف
- وجود عدد حصص أسبوعية لكل مادة
- وجود معلمين صالحين لكل مادة
- وجود قيود نصاب
- عدم وجود تناقضات تمنع التوليد

إذا فشل الفحص

- يتم إيقاف الخوارزمية

- إرجاع قائمة بالأسباب
- عدم إنشاء جدول ناقص بصمت

3 المرحلة: Build Demand Units

هذه أهم خطوة.

مستقلة Scheduling Unit بدل أن تتعامل الخوارزمية مع "مادة" بشكل عام، يجب تحويل كل احتياج إلى

مثال

يحتاج A فصل 5:

- رياضيات = 5 حصص
- علوم = 3 حصص
- عربي = 4 حصص

يتم إنشاء:

- وحدات رياضيات 5
- وحدات علوم 3
- وحدات عربي 4

يجب أن تحتوي على Unit كل

- unit_id
- class_id
- grade_id
- subject_id
- candidate_teacher_ids
- weekly_required_count
- weekly_index
- priority_weight
- constraints
- preferences

مثال بنية الوحدة:

```
{  
  
  "unit_id": "CLS12-SUB8-03",  
  
  "class_id": 12,  
  
  "grade_id": 3,  
  
  "subject_id": 8,  
  
  "candidate_teacher_ids": [15, 21],
```

```
"weekly_required_count": 5,  
  
"weekly_index": 3,  
  
"priority_weight": 92,  
  
"constraints": {  
  
    "max_per_day": 1,  
  
    "allow_consecutive": false  
  
}  
  
}
```

4 المرحلة: Prioritize Units by Difficulty

لا يجب توزيع الوحدات عشوائيًا.
يجب ترتيبها من الأصعب إلى الأسهل.

معايير الصعوبة:

- عدد المعلمين المتاحين أقل
- عدد الفترات الممكنة أقل
- المادة أساسية
- المادة لها حصص أسبوعية كثيرة
- المادة لها قيود زمنية صارمة
- الفصل مزدحم
- المعلم قريب من سقف النصاب
- المادة تتطلب غرفة خاصة

الهدف

ابدأ بالوحدات الأصعب حتى لا تبقى فقط الحالات المستحيلة في النهاية.

5 المرحلة: Build Candidate Slots

لكل وحدة، يجب إنشاء قائمة الفترات الزمنية المحتملة.

Candidate Slot =

(weekday + time_block + teacher + class)

Hard Constraints. إلا إذا اجتاز slot لكن لا يتم قبول أي

إذا Slot يتم استبعاد أي:

- المعلم مشغول
- الفصل مشغول

- الوقت غير متاح للمعلم
- الوقت غير متاح للفصل
- Teaching Slot الفترة ليست
- النصاب سيتجاوز الحد
- المادة تجاوزت الحد اليومي
- المادة ممنوعة في ذلك التوقيت
- الغرفة المطلوبة غير متاحة
- توجد حصة أخرى لنفس الفصل أو المعلم في نفس الوقت

6 المرحلة: Score Candidate Slots

Slot. بعد تكوين القائمة الصالحة مبدئيًا، يتم تقييم كل

.هذه الخطوة هي جوهر "الذكاء" في التوزيع

:معادلة التقييم العامة

Slot Score =

Availability Score

+ Distribution Score

+ Preference Score

+ Load Balance Score

+ Gap Reduction Score

+ Subject Logic Score

- Penalty Score

:مثال للأوزان

- teacher available = +30
- class available = +30
- preferred subject time = +15
- preferred teacher time = +10
- no consecutive repetition = +10
- balanced daily load = +10
- reduce teacher gap = +8
- reduce class gap = +8
- avoid heavy subject late = +7
- prevent same subject clustering = +10
- penalty for overload = -50
- penalty for poor placement = -10

النتيجة

إلى الأقل Score يتم ترتيب كل الفترات من الأعلى.

7 المرحلة: Iterative Assignment

تبدأ الخوارزمية في الإسناد الفعلي وحدة بعد أخرى.

Unit: لكل

1. خذ أعلى وحدة أولوية
2. اجلب Candidate Slots
3. Slot لكل Score احسب
4. صالح Slot اختر أعلى
5. في الجدول entry أنشئ
6. حدّث إشغال المعلم
7. حدّث إشغال الفصل
8. حدّث عداد حصص المادة في اليوم
9. حدّث عداد نصاب المعلم
10. انتقل للوحدة التالية.

مبدأ مهم

لا يتم فقط اختيار أول خانة متاحة، بل أفضل خانة ممكنة وقت التنفيذ.

8 المرحلة: Constraint Recheck After Assignment

بعد كل عملية إسناد، يجب إعادة فحص الحالة العامة.

يتم تحديث:

- teacher occupancy
- class occupancy
- daily subject counters
- teacher daily load
- teacher weekly load
- class daily density
- room occupancy مفعلة إذا كانت

هذا يمنع أن تبني الخوارزمية قرارات لاحقة على بيانات قديمة.

9 المرحلة: Fallback Strategies

إذا لم تتمكن الخوارزمية من إسناد وحدة ما، لا يجب أن تتوقف فوراً.

يجب أن تستخدم استراتيجيات معالجة بالتدرج.

الترتيب المقترح:

1. Slot جرّب ثاني أفضل
2. جرّب معلماً آخر من المرشحين

3. جرّب يومًا آخر
4. جرّب فترة أخرى أقل تفضيلاً
5. مع حصة أقل أولوية Swap فَعْل
6. محدود Backtracking نفذ
7. فقط Soft Constraints رخص بعض
8. unresolved issue إذا استمر الفشل، سجل

ممنوع تمامًا:

Hard Constraint كسر أي

10 المرحلة: Swap Logic

Swap. إذا لم يوجد مكان صالح مباشر، يمكن استخدام

مثال:

- وحدة رياضيات لا تجد مكانًا
- مناسب Slot توجد حصة نشاط في
- آخر دون ضرر Slot يمكن نقل حصة النشاط إلى
- لوحدة الرياضيات Slot يتم تحرير هذا ال

Swap شروط ال

- Hard Constraint لا يكسر أي
- لا يسبب سلسلة تعارضات غير قابلة للحل
- يفضل أن يكون مع وحدة أقل أولوية
- يتم تقييمه قبل اعتماده

11 المرحلة: Limited Backtracking

محدود، وليس شاملًا حتى لا يصبح الأداء بطيئًا Backtracking إذا استمر الفشل، تستخدم الخوارزمية

المنطق:

- ارجع عددًا محدودًا من الخطوات
- فك آخر 1 إلى 3 إسنادات مرتبطة بالتعارض
- جرّب حلولاً بديلة
- أعد البناء الجزئي

الهدف

إنقاذ الجدول دون إعادة البناء الكامل

12 المرحلة: Optimization Pass

بعد الإسناد الأولي، تبدأ مرحلة تحسين الجودة

يتم فحص

- الفجوات في جداول المعلمين
- الفجوات في جداول الفصول
- تكدس نفس المادة
- عدم توازن الأحمال
- سوء توزيع المواد الأساسية
- ازدحام أيام معينة
- ضعف بداية أو نهاية اليوم

يتم تنفيذ:

- نقل حصة من يوم مزدحم إلى يوم أخف
- تبديل حصص لتحسين التوازن
- تقليل فراغات المعلم
- تقليل تكرار المادة في نفس اليوم
- تقديم المواد الأساسية إلى فترات أفضل

13 المرحلة: Quality Evaluation

Quality Score. بعد اكتمال التوزيع والتحسين، يجب حساب

المقاييس:

- نسبة اكتمال الحصص المطلوبة
- عدد التعارضات الإلزامية
- توازن حمل المعلمين
- توازن حمل الفصول
- توزيع المواد على الأيام
- تقليل الفجوات
- احترام التفضيلات
- جودة تموضع المواد الأساسية

مثال:

Quality Score =

40% Completion

+ 20% No Hard Conflicts

+ 10% Teacher Balance

+ 10% Class Balance

+ 10% Subject Distribution

+ 10% Soft Constraints Satisfaction

14 المرحلة: Save Result as Draft Version

بعد انتهاء التوليد:

- لا يتم النشر مباشرة
- Draft يتم إنشاء نسخة
- Entries يتم حفظ جميع
- Diagnostics يتم حفظ
- يتم حفظ المشكلات
- يتم حفظ الدرجة النهائية
- كاملة Logs يتم حفظ

النتيجة

يمكن لمدير المدرسة:

- مراجعة الجدول
- اعتماد الجدول
- إعادة التوليد
- أرشفة النسخة
- العودة لنسخة سابقة

6) Hard Constraints

هذه قيود لا يمكن كسرها إطلاقاً:

1. لا يمكن أن يدرّس المعلم حصتين في نفس الوقت
2. لا يمكن أن يحضر الفصل حصتين في نفس الوقت
3. لا يمكن وضع حصة داخل الاستراحة أو الصلاة
4. لا يمكن إسناد مادة لمعلم غير مربوط بها
5. لا يمكن تجاوز النصاب اليومي للمعلم
6. لا يمكن تجاوز النصاب الأسبوعي للمعلم
7. لا يمكن استخدام وقت غير متاح للمعلم
8. لا يمكن استخدام وقت غير متاح للفصل
9. لا يمكن وضع مادة أكثر من الحد اليومي المسموح
10. لا يمكن تجاوز حدود اليوم الدراسي
11. غير نشط Slot لا يمكن استخدام
12. لا يمكن تعيين غرفة غير مناسبة إذا كانت الغرفة مفعلة

7) Soft Constraints

هذه قيود تحسين جودة:

1. توزيع المادة على عدة أيام
2. تقليل الفجوات
3. تقليل التكرار المتتالي
4. تفضيل المواد الأساسية في الصباح
5. تجنب المواد الثقيلة في آخر اليوم
6. موازنة أحمال المعلمين

7. موازنة أحمال الفصول
8. احترام تفضيلات المعلمين
9. تحسين بداية ونهاية اليوم الدراسي

قواعد توزيع ذكية يجب تطبيقها (8)

Rule 1

المواد الأساسية مثل:

- عربي
- إنجليزي
- رياضيات
- علوم

يجب أن تحصل على أولوية أعلى في الفترات الأولى.

Rule 2

إذا كانت المادة تحتاج 5 حصص أسبوعيًا، لا يجب وضعها كلها في يومين فقط إلا عند الضرورة.

Rule 3

لا تجعل نفس المادة تتكرر في فترتين متتاليتين لنفس الفصل إلا إذا كان ذلك مسموحًا صراحة.

Rule 4

تجنب أن يكون لدى المعلم فراغات طويلة غير مبررة.

Rule 5

تجنب أن يبدأ أو ينهي الفصل يومه دائمًا بمادة ثقيلة.

Rule 6

إذا تعذر الحل الكامل، أخرج أفضل حل ممكن مع تقرير واضح بالمشكلات.

أوضاع تشغيل الخوارزمية (9)

Full Generation

بناء جدول كامل من الصفر.

Partial Regeneration

إعادة توليد:

- فصل واحد
- معلم واحد
- يوم واحد
- مادة واحدة

Assist Mode

تحليل الجدول الحالي واقتراح تحسينات دون إعادة البناء الكامل.

10) Pseudocode تنفيذي

```
def generate_timetable(school_id, academic_year_id, term_id, mode="full"):
```

```
    context = load_context(school_id, academic_year_id, term_id)
```

```
    readiness_result = validate_readiness(context)
```

```
    if not readiness_result.is_valid:
```

```
        return {
```

```
            "status": "failed",
```

```
            "reason": "readiness_validation_failed",
```

```
            "issues": readiness_result.issues
```

```
        }
```

```
    demand_units = build_demand_units(context)
```

```
    prioritized_units = prioritize_units_by_difficulty(demand_units, context)
```

```
    version = create_draft_timetable_version(context, mode)
```

```
    occupancy_state = initialize_occupancy_state(context, version)
```

```
    for unit in prioritized_units:
```

```
        candidate_slots = build_candidate_slots(unit, context, occupancy_state)
```

```
        scored_slots = score_candidate_slots(unit, candidate_slots, context, occupancy_state)
```

assigned = False

for slot in scored_slots:

if passes_hard_constraints(unit, slot, context, occupancy_state):

assign_unit(unit, slot, version, occupancy_state)

assigned = True

break

if not assigned:

assigned = try_fallback_strategies(unit, version, context, occupancy_state)

if not assigned:

log_unresolved_issue(version, unit, "unassigned_unit")

run_optimization_pass(version, context, occupancy_state)

quality = evaluate_quality(version, context, occupancy_state)

save_quality_metrics(version, quality)

return {

"status": "success",

"version_id": version.id,

"quality_score": quality.total_score,

"assigned_units": quality.assigned_units,

"unassigned_units": quality.unassigned_units,

"warnings": quality.warnings

```
}
```

11) Pseudocode أدق لوظيفة الإسناد

```
def assign_unit(unit, slot, version, occupancy_state):
```

```
    create_timetable_entry(  
        version_id=version.id,  
        class_id=unit.class_id,  
        grade_id=unit.grade_id,  
        subject_id=unit.subject_id,  
        teacher_id=slot.teacher_id,  
        weekday_number=slot.weekday_number,  
        time_block_id=slot.time_block_id,  
        entry_type="lecture",  
        is_ai_generated=True  
    )
```

```
    occupancy_state.mark_teacher_busy(  
        teacher_id=slot.teacher_id,  
        weekday_number=slot.weekday_number,  
        time_block_id=slot.time_block_id  
    )
```

```
    occupancy_state.mark_class_busy(  
        class_id=unit.class_id,  
        weekday_number=slot.weekday_number,  
        time_block_id=slot.time_block_id  
    )
```

```
occupancy_state.increment_teacher_daily_load(  
    teacher_id=slot.teacher_id,  
    weekday_number=slot.weekday_number  
)
```

```
occupancy_state.increment_teacher_weekly_load(  
    teacher_id=slot.teacher_id  
)
```

```
occupancy_state.increment_subject_daily_count(  
    class_id=unit.class_id,  
    subject_id=unit.subject_id,  
    weekday_number=slot.weekday_number  
)
```

12) Pseudocode **لوظيفة التقييم**

```
def score_candidate_slot(unit, slot, context, occupancy_state):  
    score = 0  
  
    if slot.teacher_available:  
        score += 30  
  
    if slot.class_available:  
        score += 30  
  
    if is_preferred_time_for_subject(unit.subject_id, slot, context):  
        score += 15
```

if is_preferred_time_for_teacher(slot.teacher_id, slot, context):

score += 10

if not causes_consecutive_same_subject(unit, slot, occupancy_state):

score += 10

if improves_daily_balance(unit, slot, occupancy_state):

score += 10

if reduces_teacher_gaps(slot.teacher_id, slot, occupancy_state):

score += 8

if reduces_class_gaps(unit.class_id, slot, occupancy_state):

score += 8

if avoids_late_heavy_subject(unit.subject_id, slot, context):

score += 7

if causes_subject_clustering(unit, slot, occupancy_state):

score -= 10

return score

النتائج النهائي المطلوب من المطور (13)

يجب تنفيذ الخوارزمية بحيث:

- تبني الجدول بناءً على البيانات الفعلية
- تمنع التعارضات منطقيًا وقاعديًا
- توزع الحصص بذكاء لا بعشوائية
- تحترم النصاب والقيود

- تدعم إعادة التوليد الجزئي
- تسجل المشاكل والتشخيص
- قابلة للنشر لاحقًا Draft تنتج نسخة
- تكون قابلة للتوسع لاحقًا

Agent الصياغة المختصرة الجاهزة للـ (14)

Build a constraint-based timetable distribution algorithm that:

- reads all active school scheduling data from the database
- builds weekly demand units for each class-subject requirement
- sorts units by difficulty
- generates valid candidate slots
- scores candidate slots using heuristics
- assigns the best valid slot iteratively
- uses fallback strategies such as swap and limited backtracking
- runs a post-assignment optimization pass
- computes a final quality score
- saves the result as a draft timetable version with diagnostics